



CI/CD Weaponization: Build It, Deploy It, Own It





Ricardo Sanchez

- AppSec Lead
- Threat Intelligence
- pipebreach.com





@rrenatosb







Daniel Malvaceda Canales

- AppSec Architect
- DevSecOps Village Organizer
- pipebreach.com

DevSecOps Space

EKOPARTY MIAMI

Agenda

- Context (5 min)
- CI/CD (30 min)
- Break (10 min)
- Adversary Emulation (10 min)
- Hands-On Exploitation Lab (185 min)
- Takeaways
- Q&A

Objective

- Elevate GitHub offensive
- Emulate adversaries
- Springboard for future AppSec Engineers

The problem

- Security tools are frequently implemented merely for compliance rather than being genuinely equipped to detect supply chain threats.
- While standard AppSec programs heavily emphasize application-layer tools like SAST, SCA and DAST, there is a dangerous blind spot regarding CI/CD infrastructure, workflow design configurations, and malicious dependency injection occurring further down the pipeline.
- Defenders have limited visibility into how their controls perform against real-word adversary TTPs.

CI / CD



APPLICATION LAYER

SCA | SAST | DAST | Secret Scan

CI/CD Pipeline



PIPELINE INFRASTRUCTURE

EXECUTION LAYER

Workflows
Event triggers define
WHEN and HOW code runs

invokes

Actions
Third-party code
referenced by tag/SHA

runs on

Runners
Hosted or self-hosted
execution environment

runner accesses at execution time

Tokens
GITHUB_TOKEN
PATs, App tokens

Secrets
Org / Repo / Env
Injected at runtime

Cloud creds
AWS, Azure, GCP
OIDC federation

Git layer
Refs, tags, hooks

Mutable by design

Permissions

Org, repo, workflow, and job-level scopes control what each layer can reach. Default: broader than necessary. Override: must be explicit.

You secure the application. The pipeline secures itself.

Adversary Emulation

An advanced security evaluation that mimics the precise tactics, techniques, and procedures (TTPs) of active threat actors. By simulating realistic threat scenarios, it allows organizations to validate their defensive controls, identify blind spots in logging and detection, and sharpen incident response capabilities.

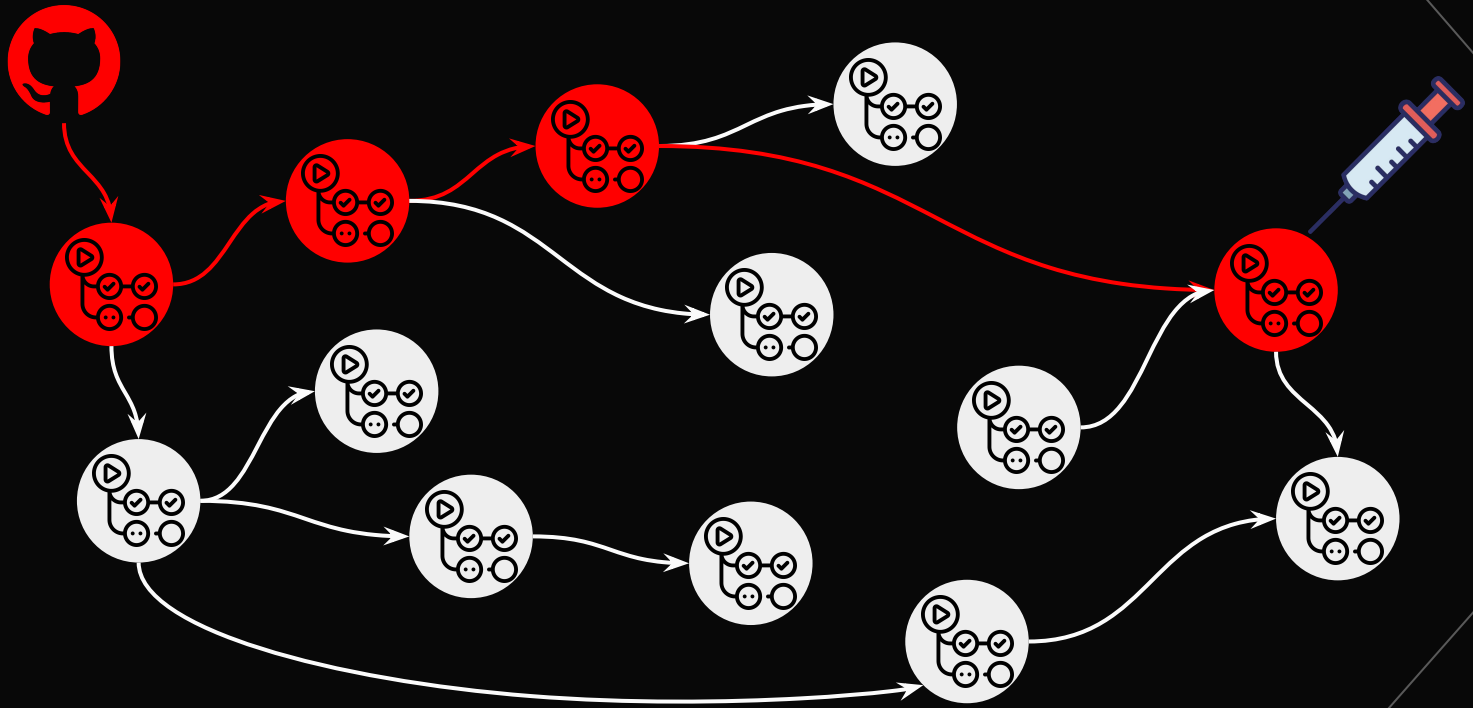
Adversary Emulation

Emulates real-world adversaries using their known TTPs.

Tests people, processes, and technology, not just tools.

Generates actionable insights to prioritize defensive improvements.

Adversary Emulation



Source → Pticha & Livni, DEF CON 32 (2024)

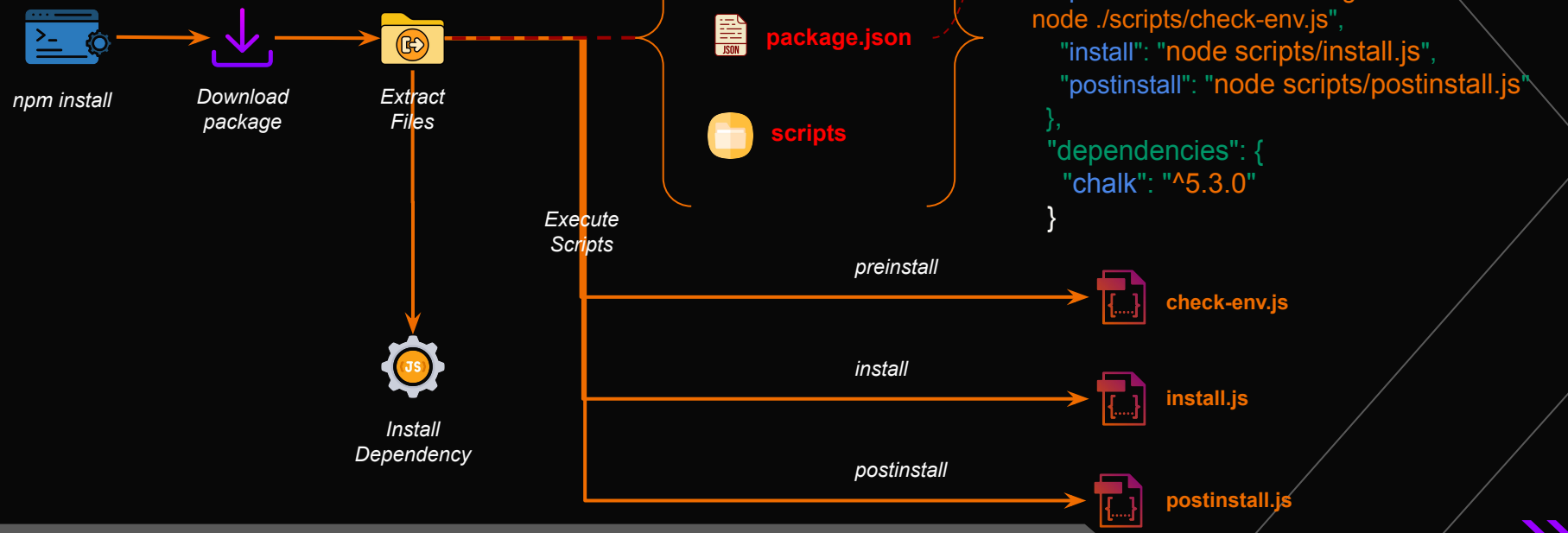
Module 1 - Initial Access (Poison Library)

“Adversaries may manipulate software dependencies and development tools prior to receipt by a final consumer for the purpose of data or system compromise. Applications often depend on external software to function properly.”

Software supply chain attack often target open-source ecosystems like PyPI, npm.

- Libraries with millions of downloads.
- Libraries with a lot of dependents.

How NPM's library work?



package.json

```
{
  "name": "my-cool-project",
  "version": "1.0.0",
  "scripts": {
    "preinstall": "echo 'Starting installation...' &&
node ./scripts/check-env.js",
    "install": "node scripts/install.js",
    "postinstall": "node scripts/postinstall.js"
  },
  "dependencies": {
    "chalk": "^5.3.0"
  }
```

Lab Module 1



We'll meet again in a moment.

TIME FOR BREAKOUT ROOMS!

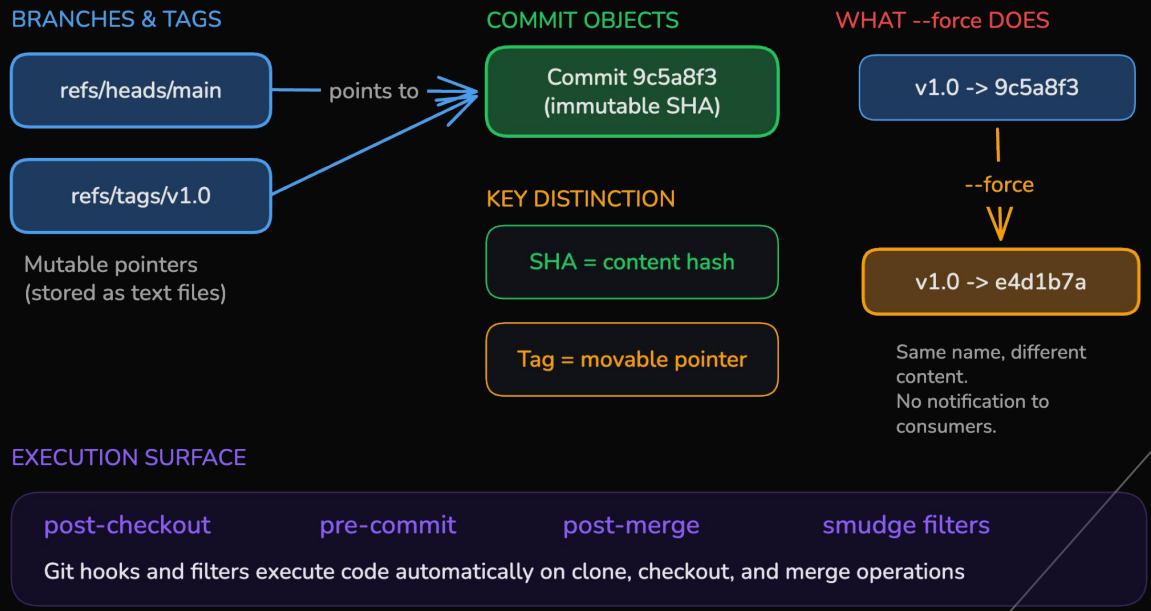
Module 2 -

- GIT
- WORKFLOWS
- ACTIONS & TAG
- PERMISSIONS
- SECRETS
- RUNNERS

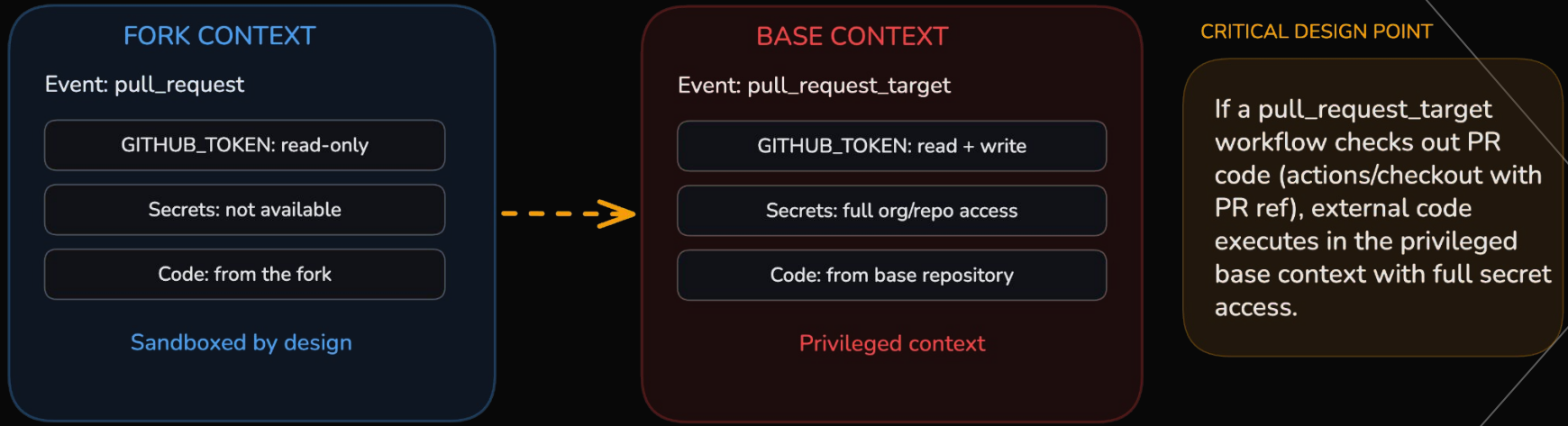
Git: refs, tags & hooks

Tags and refs are mutable pointers stored as text files under .git/refs/. Unlike a commit SHA, which is a content-addressed hash, a tag can be re-pointed at any time. Understanding this mutability is foundational to everything that follows in this module.

Tags are pointers to commits, not snapshots



Workflows & event triggers



Actions & tag references

Actions are reusable units of CI/CD logic, referenced by repository and version tag. The version is a Git tag. Since tags are mutable pointers *(previous slide)*, the code that resolves at run time can change without the reference string changing. This is by design in Git; understanding it is essential for evaluating supply chain trust.

```
# Reference by tag (standard practice)
- uses: tj-actions/changed-files@v44
  # v44 is a Git tag
  # Resolves at run time to whatever
  # commit v44 currently points to

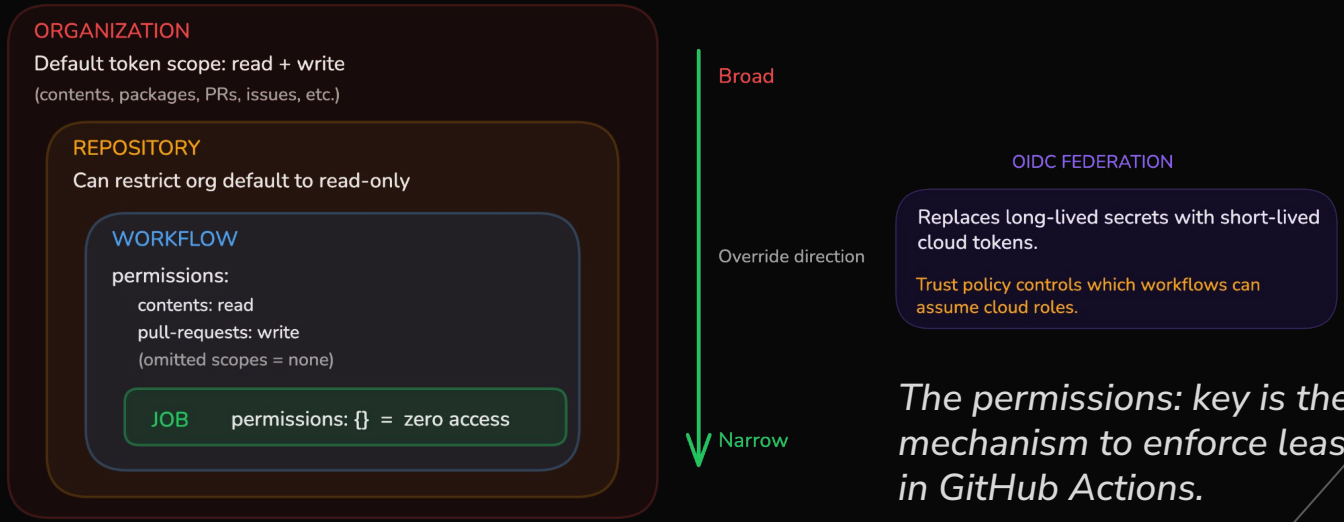
# Reference by SHA (pinned)
- uses: tj-actions/changed-files@d6babd6...
  # SHA is a content hash
  # Cannot change without changing
  # the hash itself
```

WHY THIS MATTERS

CVE-2025-30066 tj-actions/changed-files

Permissions & GITHUB_TOKEN

Every workflow run receives an automatic GITHUB_TOKEN. Its default scope is configured at the organization and repository level. The permissions key at workflow or job level is the primary mechanism to enforce least-privilege.



The permissions: key is the primary mechanism to enforce least-privilege in GitHub Actions.

Secrets, storage & masking

Secrets are encrypted at rest and injected into the runner environment at execution time. Workflow logs apply regex-based masking on the literal secret string to prevent accidental exposure in output.

```
# 1. Direct output: masked
echo "$MY_SECRET"
# Log output: ***

# 2. Encoded output: not masked
echo "$MY_SECRET" | base64
# Log output: QUtJQTNaMnhLdj...

# 3. Split output: not masked
echo "${MY_SECRET:0:10}"
echo "${MY_SECRET:10}"
# Each fragment visible in log

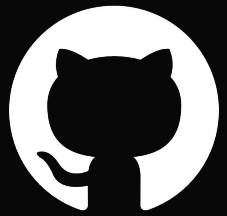
# 4. Environment inspection
cat /proc/*/environ | tr '\0' '\n'
# Lists all env vars in runner process
```

Storage hierarchy: Org, Repo, and env levels. Env secrets support protection rules such as required reviewers and wait timers.

Masking scope: The runner masks the exact literal string. Any transformation produces a string the masker does not recognize. This is a design limitation, not a bug.

Job scope: Secrets are available to all steps within the same job. Runner process memory is shared across steps and not wiped between them.

Runners, hosted vs self-hosted



GITHUB-HOSTED

Job 1 runs

↓

VM destroyed

↓

Job 2: fresh VM, no state

Ephemeral by design

Clean filesystem per job

No cross-job state leakage

Managed by GitHub

runs-on: ubuntu-latest

SELF-HOSTED

Job 1 runs

↓

State persists:
files, caches, env vars, memory

↓

Job 2: inherits prior state

Persistent by design

Prior job data on disk

Credentials in process memory

Runner groups control access

runs-on: self-hosted



Lab Module 2

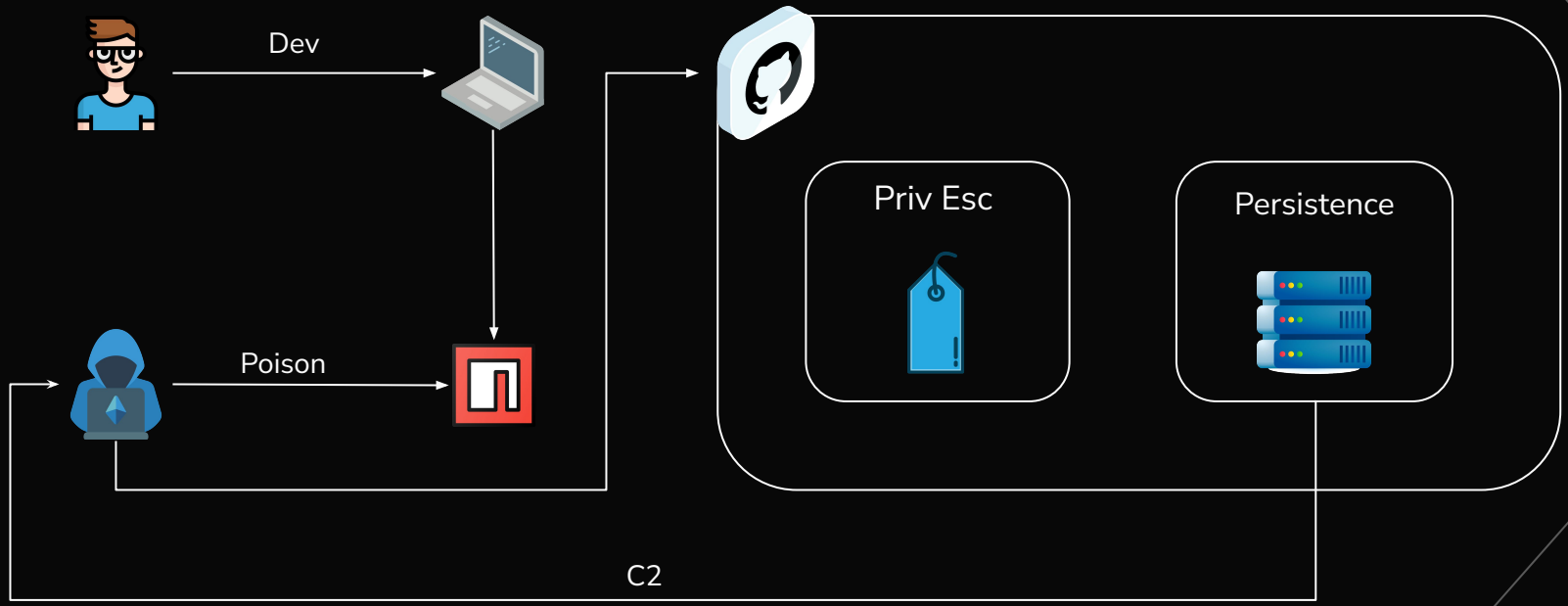


We'll meet again in a moment.

TIME FOR BREAKOUT ROOMS!

Lab Module 3

Kill Chain



Key Takeaways

Threat Intelligence come first
understanding your adversary is essential before any simulation can begin.

Context is key
We should choose the adversary, but in supply chain attacks we take care of the technology involved.

Testing should never stop
Adversary emulation should be an ongoing practice that continuously validates and improves your security posture, not a single exercise.

CI/CD and build pipelines are prime targets
It is are particularly vulnerable, often running with elevated credentials and access to production secrets.



pipebreach.com

THANK YOU!